

DecayMonitor评测平台框架v2

DecayMonitor — LLM 递归稳定性与幻觉检测评测平台 v2

2026-05-12 | 双维度版

第一部分：平台运行框架

1.1 平台定位

在 LLM 评测矩阵中新增两个正交维度：

现有评测：

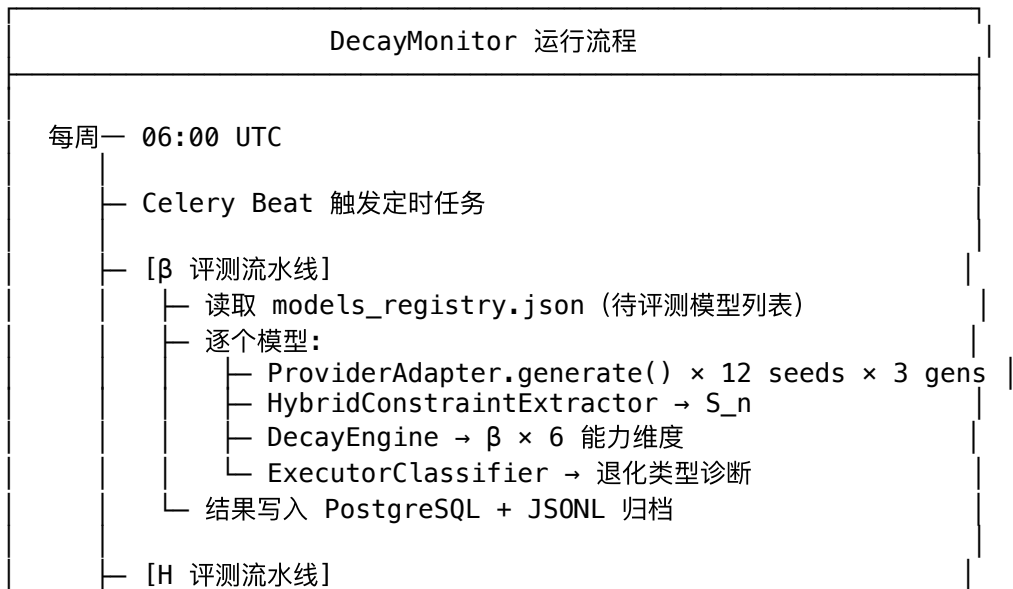
- MMLU → 知识覆盖面
- HumanEval → 代码生成能力
- Chatbot Arena → 人类偏好
- HELM → 多维度综合

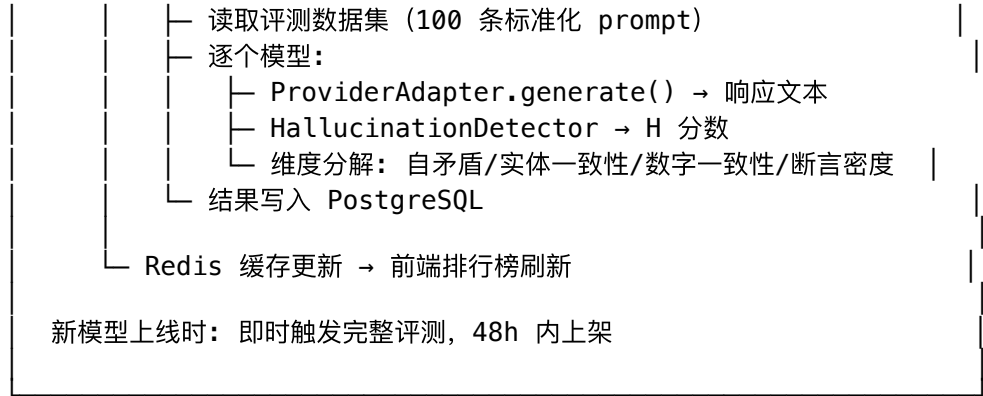
DecayMonitor新增：

- β （递归稳定性）→ "模型在自主循环中能撑多久"
- H（幻觉指数）→ "模型输出中有多少不可验证的断言"

两个维度互补： β 看稳定性，H 看可靠性。一个模型可以 β 低（稳定）但 H 高（胡说），反之亦然。开发者需要同时看两者做选型决策。

1.2 平台运行流程





1.3 模型准入管理

models_registry.json 结构:

```
{
  "models": [
    {
      "id": "claude-sonnet-4-6",
      "name": "Claude Sonnet 4.6",
      "family": "Claude",
      "provider": "quickrouter",
      "status": "active",
      "added": "2026-05-01",
      "last_evaluated": "2026-05-12",
      "beta": 0.3106,
      "h_score": 0.12,
      "tags": ["commercial", "text-only"]
    }
  ]
}
```

准入规则: – 模型必须可通过公开 API 访问 (保证评测可复现) – 开源模型需提供 HuggingFace 链接 + 推理配置 – 模型更新 (同 ID 新版本) 自动触发复测

1.4 排行榜展示设计

首页 = 双维度总榜:

排名	模型	β↓	H↓	数学	代码	事实	逻辑	创意	通用
1	Qwen2.5-7B	0.29	0.15	●	●	●	●	●	●
2	Claude Opus 4.6	0.30	0.09	●	●	●	●	●	●
3	DeepSeek-V3	0.31	0.11	●	●	●	●	●	●
4	Claude Sonnet 4.6	0.31	0.08	●	●	●	●	●	●
5	GPT-4o	0.34	0.10	●	●	●	●	●	●
...									

颜色编码: ● 优秀(<0.15) ● 良好(<0.25) ● 一般(<0.35) ● 差(>0.35)

模型详情页 = 完整画像: – β 部分: 6 维衰减曲线 + S_n 跨代轨迹 + 执行者退化诊断 – H 部分: 4 维幻觉雷达图 + 典型幻觉案例 + 风险分布 – 综合: β-H 四象限定位图 (稳定+可靠 / 稳定+撒谎 / 退化+可靠 / 退化+撒谎)

模型对比页： – 选 2-4 模型，并排对比全部指标 – 自动生成一句话结论：“模型 A 比模型 B 更稳定 (β 低 15%)，但幻觉风险更高 (H 高 40%)”

1.5 质量保证

机制	频率	说明
评测复现性检查	每模型每次	同一模型两次评测 β 偏差 < 0.03
校准数据集验证	月度	用固定校准集验证 H 检测器漂移
β 历史趋势监控	周度	同模型 β 突然跳变 → 人工复核
社区反馈通道	持续	GitHub Issue 接收模型厂商申诉

第二部分：商业框架

2.1 价值主张（双维度）

现有的 LLM 评测告诉你模型”答对多少题”、“人类多喜欢”。我们告诉你两件事： β — 模型在 agent 循环中能撑多久 H — 模型输出里有多少东西是编的

对 AI agent 开发者，这两个指标直接决定”用哪个模型跑我的 agent”。

2.2 收入来源（三档）

	Free	Pro	Enterprise
价格	\$0	\$1,499/次	\$4,999/月
β 评测	12 seeds × 3 gens	100 seeds × 5 gens	自定义配置
H 评测	100 条标准 prompt	1000 条 + 自定义领域	完全自定义
结果公开	上榜	闭门（不上榜）	闭门
执行者诊断	基本	深度 E-I/E-II/E-III	深度 + 训练建议
幻觉案例分析	摘要	完整案例 + 类型分布	完整 + 定制报告
竞品对比	公开数据	匿名对比	指定竞品对比
API 访问	50 次/天	包含	无限
历史追踪	否	否	✅ β +H 历史曲线
训练数据诊断	否	否	✅ 数据盲区定位
交付物	—	PDF + CSV/JSONL	PDF + API + Slack
客户	所有人	模型厂商发布前	训练团队每 checkpoint

定价逻辑： LMArena 定制评测 \$5,000–50,000/次，我们 \$1,499 是初期获客价。首 10 个客户后上调至 \$2,999。

2.3 目标客户画像

第一优先级：模型厂商（发布前评测） – 人物：LLM 厂商的 evaluation 负责人 / 产品经理 – 场景：新版本发布前，需要除 benchmark 之外的独立第三方数据 – 痛点：“MMLU 分很高但 beta 用户反馈 agent 场景表现不稳定，找不到原因” – 我们提供：β+H 作为发布前 checklist 的两个新维度 – 获客方式：论文发表后定向联系 Anthropic/DeepSeek/01.AI 的 eval 团队

第二优先级：AI agent 初创公司（选型决策） – 人物：CTO / 技术负责人 – 场景：“我要做客服 agent，GPT-4o 还是 Claude Sonnet” – 痛点：benchmark 数据没区分度，实际 agent 场景表现跟 benchmark 不一致 – 我们提供：β 排名 + H 风险评级 + 场景化对比 – 获客方式：内容营销（“Agent 选型指南”系列）+ HuggingFace demo

第三优先级：AI 投资/研究机构 – 人物：分析师 / 研究员 – 场景：需要客观的模型稳定性量化数据 – 我们提供：β 历史趋势 + 跨家族对比

2.4 收入预测

时间	模型数 Pro/月 Enterprise			月收入	状态
Month 1–3	4→10	0	0	\$0	Phase B/C: 论文+开源
Month 4–6	15	2–3	0	\$3,000–4,500	网站上线，早期 Pro
Month 7–9	20	5	1	\$12,500	口碑传播
Month 10–12	25	8	2	\$22,000	稳定增长
Month 13–18	35	12	4	\$38,000	品牌建立
Month 19–24	50	18	6	\$57,000	

2.5 成本结构

开发成本（一次性）：

项目	费用
网站开发（前后端）	自有时间
论文投稿费	\$0（arXiv 免费）
设计 + Logo	\$500（一次性外包）

运营成本（月度）：

项目	月费
云服务器（4 vCPU + 8GB）	\$60
PostgreSQL managed	\$30
Redis	\$15
域名 + CDN	\$20
β 评测 API 费用（30 模型 × \$0.50）	\$15
H 评测 API 费用（30 模型 × \$1.00）	\$30
合计	\$170

盈亏平衡点： 月收入 > \$170，即每月 1 个 Pro 客户即可覆盖全部运营成本。

2.6 竞争壁垒

壁垒	性质	持久性
β 定义权 + 理论体系（约束吸引子坍塌）	学术 + 品牌	长期
H 检测算法（纯文本特征，CPU only）	技术	中短期
跨家族 β+H 数据库（先发数据优势）	数据	中长期
多 Provider 抽象层（快速接入新模型）	工程	短期
独立第三方立场（不收模型厂商赞助）	品牌	长期（最难复制）

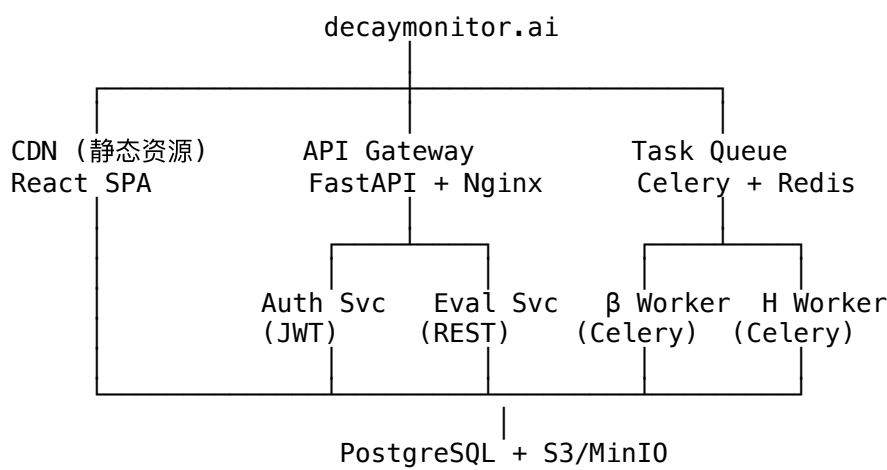
最难复制的：独立第三方信任。 LMArena 的争议就在于”既当裁判又收参赛者钱”。我们从一开始就不拿模型厂商的钱——Pro/Enterprise 是评测服务费，不是赞助费。排行榜上的模型排序完全独立。

2.7 退出策略

路径	概率	预期结果
被 LMArena/HuggingFace/Weights&Biases 收购	中	\$3–10M
独立运营，小团队自给自足	高	2–3 人团队，\$30–50k MRR
融资规模化	低	需证明 TAM > \$50M

第三部分：技术实现框架

3.1 系统架构总览



评测引擎核心：
ProviderAdapter → ExperimentRunner → DecayEngine → ExecutorClassifier
→ HallucinationDetector (new)

3.2 β 评测算法（已有，完整保留）

3.2.1 文本特征提取

对每个样本，纯规则提取 8 个表面统计特征（CPU only，零模型依赖）：

E-I 公理级（逻辑/推理链）：

$ei_logic_density = \text{count}(\text{逻辑连接词} \in \text{LOGIC_CONNECTORS}) / n_tokens$

LOGIC_CONNECTORS = {therefore, because, thus, hence, consequently,
moreover, furthermore, nevertheless, otherwise,
因此, 所以, 然而, 因为, 从而, 于是, 故, 则, ...}

$ei_syntax_cv = \text{std}(\text{句子token数}) / \text{mean}(\text{句子token数})$

E-II 标度级（风格/表达）：

$eii_bigram_repetition = \text{count}(\text{重复的相邻词对}) / (\text{total_bigrams} + 1)$

$eii_unique_word_ratio = \text{len}(\text{set}(\text{words})) / \text{len}(\text{words})$

FILLER_WORDS = {the, a, is, of, it, in, this, that, for, with, on, as, at, by}
 $eii_filler_ratio = \text{count}(w \in \text{FILLER_WORDS}) / n_words$

$eii_truncation_ratio = \text{count}(\text{len}(w) > 7 \text{ 且被截断}) / \text{count}(\text{len}(w) > 7)$

E-III 边界级（事实/格式）：

$eiii_proper_case_ratio = \text{count}(\text{首字母大写词}) / n_words$ （英文）
 $= \text{count}(\text{专名标注}) / n_entities$ （中文）

$eiii_number_integrity = \text{count}(\text{未变化的数字实例}) / \text{count}(\text{数字实例})$

自适应分块：长文本（>300 tokens）自动切片到 Goldilocks 区（80–200 tokens），各块特征取均值。

3.2.2 特征 → 约束状态

$\sigma_{syntax} = 0.65 \times ei_logic_density + 0.35 \times ei_syntax_cv$
 $\sigma_{style} = unique_word_ratio \times (1 - 0.5 \times bigram_rep - 0.3 \times trunc - 0.2 \times filler)$
 $\sigma_{fact} = 0.55 \times proper_case_ratio + 0.45 \times number_integrity$
 $\sigma_{coherence} = 0.55 \times ei_logic_density + 0.45 \times (1 - bigram_rep)$
 $\sigma_{safety} = 0.5$

$S_n = (\sigma_{fact} + \sigma_{syntax} + \sigma_{style} + \sigma_{safety} + \sigma_{coherence}) / 5$

3.2.3 S_n 序列 → β

指数衰减模型： $S_n = S_0 \times (1 - \beta)^n$

对数线性化： $\log(S_n / S_0) = n \times \log(1 - \beta)$

最小二乘线性回归：

$x = [1, 2, 3]$ （代数）
 $y = [\log(S_1/S_0), \log(S_2/S_0), \log(S_3/S_0)]$

$slope = \Sigma(x_i - \bar{x})(y_i - \bar{y}) / \Sigma(x_i - \bar{x})^2$
 $\beta = 1 - e^{slope}$

β 钳制在 $[0.03, 0.55]$

全局 $\beta = \text{mean}(\beta_{\text{math}}, \beta_{\text{code}}, \beta_{\text{fact}}, \beta_{\text{logic}}, \beta_{\text{creative}}, \beta_{\text{general}})$

3.3 幻觉检测算法（新增）

3.3.1 设计原则

真正的幻觉检测需要外部知识库做事实核验——这不切实际（覆盖面、维护成本、多语言）。我们的方案是测”幻觉风险”而非”幻觉事实”——识别文本中提示幻觉可能性的信号。

四个子维度，全部纯文本规则，不依赖外部知识：

3.3.2 维度一：自矛盾检测（内部一致性）

算法：将文本拆分为 N 条断言 $\rightarrow$ 逐对比较语义冲突

实现：

1. 用句号/分号/换行切分成句子
2. 对每个句子提取"主语-谓语-宾语"三元组（spaCy / 规则）
3. 构建三元组集合 $T = \{(s, p, o)\}$
4. 检测矛盾对：
 - 直接冲突： (s, p, o) 和 $(s, \text{NOT } p, o)$ 或数值冲突
 - 隐含冲突：两个三元组推理后矛盾

矛盾分数 = $\text{count}(\text{矛盾对}) / C(n_{\text{sentences}}, 2)$ # 归一化

示例：

```
"Paris is the capital of France."  
→ (Paris, is_capital_of, France)  
  
"The capital of France is Lyon."  
→ (capital_of_France, is, Lyon)  
→ 冲突: Paris ≠ Lyon → 得分 +1
```

简化实现 (v1)： 不提取完整三元组，用数值冲突 + 实体指代一致性做近似： - 扫描所有数字，检测同一上下文中数字冲突（“5 million...50 million”） - 扫描所有命名实体，检测同一实体的属性冲突

3.3.3 维度二：实体一致性

算法：追踪文本中每个命名实体的所有引用，检测属性漂移

实现：

1. NER 提取（spaCy / 规则）
2. 对每个实体 E ，收集所有谓词 $P_i(E)$
3. 检测：对同一属性是否有多个不同值

实体一致性 = $1 - (\text{属性冲突的实体数} / \text{实体总数})$

示例：

```
"John Smith, 45, a doctor... John, 52, works at the hospital..."  
→ 实体 "John Smith" 的年龄属性冲突: 45 ≠ 52
```

3.3.4 维度三：数字一致性

算法：扫描文本中所有数字，检测逻辑矛盾

实现：

- 1. 提取所有（数值，单位，上下文窗口）元组
- 2. 分组检测：
 - 总和一致："A占30%，B占40%，C占20%" → 30+40+20=90≠100
 - 时序一致："2020年公司成立，2015年推出首款产品" → 时序倒错
 - 比率一致：同一实体的数值在不同位置是否自洽

数字一致性 = 1 - （矛盾数字对 / 数字对总数）

3.3.5 维度四：断言密度（模糊性指标）

算法：测量文本中"强断言"vs"模糊表达"的比例

强断言标记：

"is", "was", "will", "always", "never", "all", "none"
"研究表明", "数据证明", "事实是"

模糊表达标记：

"may", "might", "could", "suggests", "possibly", "arguably"
"一般认为", "有观点认为", "据传"

断言密度 = count(强断言标记) / (count(强断言标记) + count(模糊表达标记))

高断言密度 ≠ 幻觉。但当断言密度高 + 其他维度得分低时 → 高风险：
模型对不确定的事情过于自信

3.3.6 幻觉指数 H 计算

H_raw = w1 × (1 - 自矛盾分) + w2 × 实体一致性 + w3 × 数字一致性

其中 w1=0.40, w2=0.35, w3=0.25 (权重经 LLM judge 校准)

H_risk_adj = H_raw × (1 + 0.3 × 断言密度)
断言密度作为风险放大器：太自信的模型如果错了，后果更严重

H = 1 - H_risk_adj # 最终幻觉指数 ∈ [0,1]，越低越好

H 分级：

H 值	评级	含义
< 0.10	极可靠	自洽性高，实体/数字一致
0.10–0.20	较可靠	偶有矛盾，agent 场景可用
0.20–0.35	有风险	需人工审核输出
> 0.35	高风险	高频自矛盾，不宜自主决策

3.3.7 与 β 的关系

β 和 H 测的是不同东西：

- β = 递归退化速率（跨代测量，需要递归生成）
- H = 单次幻觉风险（单次输出即可测量）

交叉分析：

β 低，H低 ★ 最佳agent	β 高，H低 退化快但不说谎
β 低，H高 稳定但爱编造	β 高，H高 ★ 不宜agent

3.4 数据库设计

-- 模型注册表

```
CREATE TABLE models (  
  id          VARCHAR(100) PRIMARY KEY,  
  name        VARCHAR(200) NOT NULL,  
  family      VARCHAR(50),  
  provider    VARCHAR(50) NOT NULL,  
  provider_model VARCHAR(200) NOT NULL,  
  status      VARCHAR(20) DEFAULT 'active',  
  created_at  TIMESTAMP DEFAULT NOW()  
);
```

-- β 评测结果

```
CREATE TABLE beta_results (  
  id          SERIAL PRIMARY KEY,  
  model_id    VARCHAR(100) REFERENCES models(id),  
  evaluated_at TIMESTAMP DEFAULT NOW(),  
  global_beta FLOAT NOT NULL,  
  beta_math   FLOAT,  
  beta_code   FLOAT,  
  beta_fact   FLOAT,  
  beta_logic  FLOAT,  
  beta_creative FLOAT,  
  beta_general FLOAT,  
  n_seeds     INT DEFAULT 12,  
  n_generations INT DEFAULT 3,  
  lineage_path VARCHAR(500), -- JSONL 归档路径  
  diagnosis   JSONB, -- 执行者退化诊断  
  meta        JSONB  
);
```

-- H 评测结果

```
CREATE TABLE hallucination_results (  
  id          SERIAL PRIMARY KEY,  
  model_id    VARCHAR(100) REFERENCES models(id),  
  evaluated_at TIMESTAMP DEFAULT NOW(),  
  h_score     FLOAT NOT NULL,  
  h_self_contra FLOAT, -- 自矛盾分维  
  h_entity_consist FLOAT, -- 实体一致性  
  h_number_consist FLOAT, -- 数字一致性  
  h_assert_density FLOAT, -- 断言密度  
  n_prompts   INT DEFAULT 100,  
  detail_path VARCHAR(500), -- 详细结果归档  
);
```

```

    meta                JSONB
);

-- 用户表
CREATE TABLE users (
    id                  SERIAL PRIMARY KEY,
    email               VARCHAR(255) UNIQUE,
    name                VARCHAR(100),
    avatar_url          VARCHAR(500),
    auth_provider        VARCHAR(20),          -- 'github', 'google', 'email'
    auth_id              VARCHAR(100),
    plan                VARCHAR(20) DEFAULT 'free',
    pro_credits          INT DEFAULT 0,
    stripe_customer_id   VARCHAR(100),
    created_at           TIMESTAMP DEFAULT NOW(),
    last_login           TIMESTAMP
);

-- 评测请求 (Pro 用户提交)
CREATE TABLE eval_requests (
    id                  SERIAL PRIMARY KEY,
    user_id             INT REFERENCES users(id),
    model_name           VARCHAR(200),
    eval_type            VARCHAR(20),          -- 'beta', 'hallucination', 'both'
    status               VARCHAR(20) DEFAULT 'pending',
    result_beta          FLOAT,
    result_h             FLOAT,
    report_path          VARCHAR(500),
    requested_at         TIMESTAMP DEFAULT NOW(),
    completed_at         TIMESTAMP
);

```

3.5 API 设计

公开 API (无需认证):

```

GET /api/v1/leaderboard?sort=beta&limit=20
GET /api/v1/models/{model_id}
GET /api/v1/models/{model_id}/beta-history
GET /api/v1/models/{model_id}/hallucination-detail
POST /api/v1/compare { models: [...], dimensions: [...] }

```

认证 API (需 JWT):

```

POST /api/v1/auth/github      { code }
POST /api/v1/auth/google     { code }
POST /api/v1/auth/register   { email, password }
POST /api/v1/auth/login      { email, password }

```

Pro API (需 Pro+ 订阅):

```

POST /api/v1/eval/private     { model, type, seeds, custom_prompts? }
GET /api/v1/eval/status/{request_id}
GET /api/v1/eval/download/{request_id}

```

Enterprise API (需 Enterprise 订阅):

```

POST /api/v1/eval/continuous { model, schedule, webhook_url }
GET /api/v1/enterprise/dashboard

```

3.6 技术栈

层	技术选型	理由
前端	React + Next.js + Tailwind	生态成熟，SEO 友好（Next.js SSR）
图表	ECharts / D3.js	6 维雷达图 + 衰减曲线，中文友好
后端	FastAPI (Python)	与现有评测引擎同语言，零桥接
任务队列	Celery + Redis	Python 原生，已有集成基础
数据库	PostgreSQL	JSONB 支持，评测数据结构灵活
文件存储	S3 / MinIO	JSONL 归档 + 报告 PDF
认证	NextAuth.js + python-jose	GitHub OAuth 一行配置
支付	Stripe	标准方案，Checkout 几行代码
部署	Docker + GitHub Actions CI	可迁移，不锁定云厂商
监控	Sentry + Uptime Robot	免费层足够

3.7 开发阶段

Phase D-1: 网站 MVP（4 周）

周	任务
W1	FastAPI 骨架 + 数据库 schema + 评测引擎对接
W2	排行榜首页 + 模型详情页（静态数据，4 模型）
W3	模型对比页 + 图表交互
W4	Celery 定时评测 + 认证系统 + 部署上线

Phase D-2: 付费功能（4 周）

周	任务
W1	Pro 评测流程 + 报告生成
W2	Stripe 集成 + 权限控制
W3	Enterprise dashboard + 历史追踪
W4	测试 + 安全审计 + 灰度上线

总开发量：8 周（1 人全职）或 4 周（2 人并行）。

第四部分：幻觉检测的技术验证计划

4.1 已有基础

与 β 评测共享基础设施： – ProviderAdapter 多后端 → 调用模型获取评测文本 – HybridConstraintExtractor → 已有 E-III 退化检测（数字/大小写） – ExperimentRunner 管线 → 批

量评测自动化

新增部分仅需 HallucinationDetector 类 + 评测 prompt 数据集。

4.2 算法校准计划

- 1. 构造校准数据集：100 条标准 prompt (50 条事实型 + 50 条推理型) × 4 模型 = 400 条样本
- 2. 人工标注：每条样本标注自矛盾/实体错误/数字错误 (二元判断)
- 3. 算法优化：调整三权重 w1/w2/w3 使 F1 最大化
- 4. LLM Judge 验证：用 Claude 做幻觉判断，计算 Spearman ρ (与 β 的 0.93/0.98 对标)

4.3 与 β 评测的协同效应

- 同一段 API 调用产出两个指标：一次 generate() 的 text 可以同时输入 β 管线 (作为 Gen1 数据) 和 H 管线 (作为单次输出)
- 评测成本几乎不增：β 评测是 12 seeds × 3 gens = 36 次调用，H 评测是 100 次调用。总共 136 次/模型。H 评测的 100 次调用中取 12 条作为 β 的 seeds，复用。
- 联合效用：β 低 + H 低 = 最佳 agent 模型。这个组合标签是唯一性的。

第五部分：当前状态与启动条件

5.1 已完成

资产	状态
β 算法 + 4 模型数据库	✓
ExperimentRunner 统一评测框架	✓
ProviderAdapter 多后端	✓
纯文本特征提取器 (CPU only)	✓
产品方向确认 (评测平台)	✓
商业模型框架	✓

5.2 待启动

优先级	任务	前置条件	费用
P0	DeepSeek-V3 β 实验 (第三家族)	API key	\$5
P0	P3 严格检验	GPU 机器	\$0
P0	幻觉检测算法原型	0 依赖	\$0
P1	n=100 seeds 统计验证	API 费用	\$126
P1	Llama 3 β 实验 (第四家族)	Together API	\$10
P2	H 评测校准数据集构造	0 依赖	\$0

优先级	任务	前置条件	费用
P2	论文初稿	P0+P1 数据	\$0
P3	网站 MVP 开发	论文提交后	自有时间
P3	付费功能开发	有 Pro 意向客户后	自有时间

5.3 可立即启动的

本周即可，零外部依赖：

- 1. 幻觉检测算法 v1 — HallucinationDetector 类
 - 自矛盾检测（数值冲突 + 实体属性冲突）
 - 实体一致性（NER + 属性追踪）
 - 数字一致性（总和/时序/比率）
 - 断言密度
 - 在已有 4 模型 lineage 数据上验证
- 2. H 评测 prompt 集设计 — 100 条标准化 prompt
 - 50 条事实型（触发幻觉倾向）
 - 50 条推理型（触发自矛盾）
 - 多样化的领域覆盖
- 3. DeepSeek API 接入 — 注册 + 跑第一个 β 实验